

DSA Problem Solving Patterns

Sliding Window

-> Used for problems dealing with contiguous subarrays or substrings. Ex: Max sum of subarray of size k, Longest substring without repeating characters.

Two Pointers

-> Used when iterating from both ends. Ex: Reverse string, 2-sum in sorted array.

Fast and Slow Pointers

-> Detect cycles in linked list/array. Ex: Detect loop, Happy number.

Prefix Sum

-> Used for range sum queries. Ex: Range sum, Subarray sum equals k.

Hashing

-> Frequency counting or quick lookup. Ex: Two Sum, Group Anagrams.

Binary Search

-> Search in sorted arrays or monotonic functions. Ex: Rotated array search, First/Last occurrence.

Recursion / Backtracking

-> Explore all possibilities. Ex: N-Queens, Subsets.

Dynamic Programming

-> Overlapping subproblems & optimal substructure. Ex: Fibonacci, LCS.

DFS / BFS

-> Used in graph/tree traversal. Ex: Number of islands, Clone graph.

Union-Find

-> Used for connectivity. Ex: Connected components, Kruskals Algorithm.

Bit Manipulation

-> Efficient integer operations. Ex: Count set bits, Power of 2.

Greedy

-> Make local optimal choices. Ex: Jump game, Interval scheduling.

Stack

-> Next greater/smaller element, expression parsing. Ex: Valid parentheses, Histogram area.

Topological Sort

-> Used in DAG problems. Ex: Course Schedule, Alien Dictionary.

Trie

-> Efficient prefix-based operations. Ex: Implement Trie, Word search.

Heap

-> Get min/max efficiently. Ex: Top K elements, Median in stream.